

Week 5: Apache Spark Fundamentals & Data Processing

This repository contains the theory and practical implementation focusing on Apache Spark fundamentals, data cleaning, transformations, and aggregation pipelines using PySpark.

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Answer: MapReduce suffers from disk-I/O bottlenecks due to writing intermediate states to HDFS, has high latency for iterative jobs, and uses a rigid two-stage programming model. Spark is preferred because it uses in-memory computing, enables efficient DAG optimization, and significantly lowers latency.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Answer: Spark caches datasets in RAM across cluster nodes. This prevents the costly process of serializing data to disk between iterations, allowing iterative algorithms to run entirely in memory after the initial read, often resulting in speedups of up to 100x.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

Answer:

Python

```
df_deduped = df.dropDuplicates(["user_id", "transaction_date"])
```

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

Answer:

Python

```
result = df_sales.filter(F.col("region") == "West") \
    .groupBy("product_category") \
    .agg(F.avg("sale_amount").alias("average_sale_amount"))
```

Q5: What is the difference between .na.drop() and .na.fill()? Provide a code example of filling null values in a status column with the string 'Unknown'.

Answer: .na.drop() removes the entire row if a null is found, while .na.fill() replaces the null value with a default.

Python

```
df = df.na.fill({"status": "Unknown"})
```

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Answer:

Python

```
df.groupBy("city").count().filter(F.col("count") > 100)
```

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Answer: Since DataFrames are immutable, operations don't change them in place; they

return a new DataFrame reference. This supports fault tolerance through Lineage/DAG tracking, allowing Spark to recompute lost partitions if a node fails.

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Answer:

Python

```
df.filter((F.col("age").between(18, 30)) & (F.col("subscription") == "Premium"))
```

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Answer: Null values can lead to unpredictable mathematical results or runtime errors. Handling them early (e.g., using .na.fill(0)) ensures mathematical integrity and prevents aggregation failures.

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

Answer:

Python

```
df = df.withColumn("event_time", F.col("raw_timestamp").cast(TimestampType())) \
    .drop("raw_timestamp")
```

Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Answer: Shuffling redistributes data across the network so identical keys reside on the same worker node. It is "wide" because the output partition depends on inputs from multiple input partitions, necessitating heavy network/disk I/O.

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

Answer:

Python

```
df = df.filter(F.col("email").isNotNull() & (F.col("username") != ""))
```

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Answer:

Python

```
df.agg(F.min("price").alias("min_price"),
      F.max("price").alias("max_price"),
      F.mean("price").alias("mean_price"))
```

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

Answer: It forces an expensive extra pass over the data. If date formats are inconsistent, Spark may incorrectly default the column to StringType, preventing the use of optimized date/time functions later in the pipeline.

Q15: Write a final processing pipeline that filters out duplicates, fills null prices with 0, and groups by store_id to calculate total revenue.

Answer:

Python

```
final_df = df.dropDuplicates() \
               .na.fill({"price": 0.0}) \
               .groupBy("store_id") \
               .agg(F.sum("price").alias("total_revenue"))
```

📁 Repository Structure

- * `spark.ipynb`: The executable PySpark pipeline.
- * `spark\_assignment\_dataset.csv`: The dataset used to perform actions.
- * `spark\_theory\_fundamentals.txt`: Theory, Architecture & Data Cleaning.
- * `README.md`: Project documentation and theory.